

Analyse de LunarLander

Alex Maloigne et Kamel Nehlil

1 Présentation de LunarLander

Lunar Lander est un environnement d'apprentissage par renforcement de Gym, simulant l'atterrissage d'un module spatial sur une surface lunaire à l'aide du moteur physique Box2D. L'agent doit contrôler ses moteurs afin d'atteindre une zone d'atterrissage tout en minimisant sa vitesse et sa consommation de carburant. L'espace d'observation est composé de 8 variables, incluant la position, la vitesse et l'orientation du lander. L'environnement propose un espace d'actions discret, comprenant 4 actions possibles : ne rien faire, activer le moteur principal, activer le moteur latéral gauche ou activer le moteur latéral droit. La fonction de récompense encourage un atterrissage précis et stable, tout en pénalisant les crashes. Cet environnement sera utilisé pour tester l'algorithme DQN dans un cadre d'apprentissage avec des observations partielles.

2 Analyse des wrappers basiques

Nous avons 4 courbes :

- En rouge, DQN sans x point, y point et theta point (nommé Van).
- En bleue, DQN sans x point, y point et theta point mais avec en mémoire la valeur précédente de théta (nommé Theta Ext).
- En cyan, DQN sans x point, y point et theta point mais avec en mémoire la valeur précédente de x et y (nommé XY Ext).
- En noir, DQN sans x point, y point et theta point mais avec en mémoire la valeur précédente de x,y et théta (nommé Theta XT Ext).

Il est à noter que les courbes susnommées se verront attribuer les mêmes couleurs pour tous les graphiques.

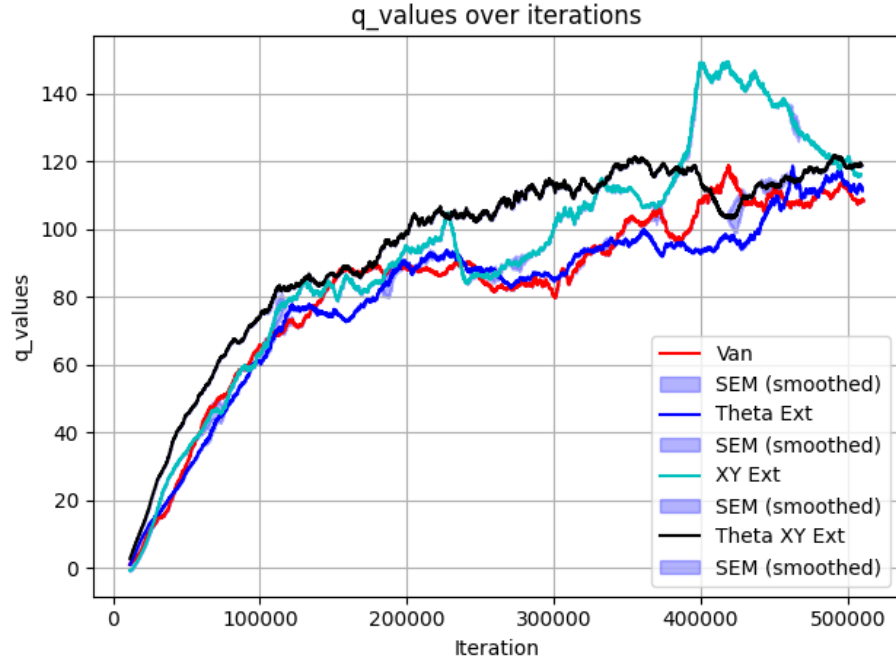


Figure 1: Valeur moyenne des q valeurs

On observe que les Q-values augmentent plus rapidement lorsque l'agent dispose d'une mémoire lui permettant d'estimer les observations manquantes, par rapport à un apprentissage sans mémoire. Dans Lunar Lander, l'objectif repose principalement sur le contrôle de l'orientation et de la vitesse d'atterrissage, plutôt que sur la position absolue. Ainsi, les paramètres liés à la position et à la vitesse verticale sont plus utiles en mémoire que l'angle spatiale du lander. Les Q-values montrent alors une meilleure convergence et une plus grande facilité à maximiser la récompense lorsque ces informations sont prises en compte. L'observation de ces paramètres clés permet de mieux résoudre le problème du monde partiellement observable, car l'agent peut ainsi estimer les variables manquantes et adopter une stratégie d'atterrissage plus stable et efficace.

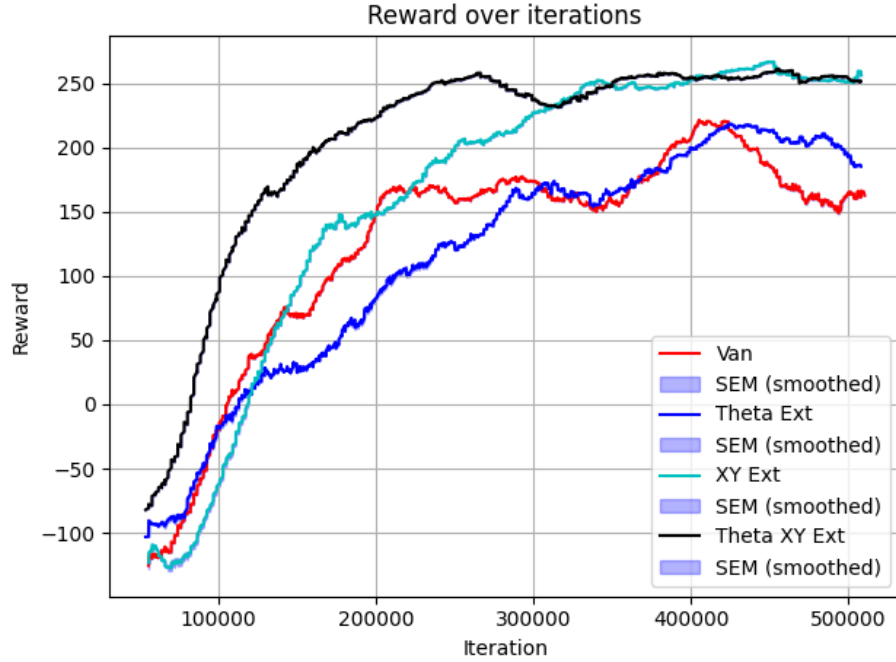


Figure 2: Valeur moyenne des rewards

Comme le montre le graphique ci-dessus, les résultats varient considérablement en fonction de la mémoire utilisée. Initialement, la courbe sans mémoire reste relativement constante après avoir atteint un certain seuil de récompense, mais commence à baisser vers la fin. On peut supposer que cela est dû à un phénomène de surapprentissage ou à une perte d'information contextuelle, ce qui fait que le modèle ne parvient plus à s'améliorer. L'ajout de la mémoire des positions (x , y) entraîne, quant à elle, une croissance marquée. On remarque la même chose pour l'ajout de la mémoire angulaire. La fusion des deux mémoires, quant à elle, converge beaucoup plus rapidement. Ces trois mémoires atteignent un reward de 250, ce qui montre qu'elles réussissent à atterrir, la seule différence étant la vitesse de convergence.

3 Analyse de Lunar Lander avec l'action Wrapper

Dans cette partie, nous allons expérimenter avec l'Action Wrapper et ses effets potentiellement positifs. Nous testerons 4 scénarios différents dans l'environnement partiellement observable défini plus haut (sans x,y et sans θ):

- Sans mémoire

- Avec mémoire des positions (x,y)
- Avec mémoire sur théta
- Avec mémoire combinée des positions et de théta
- Sans wrappers

Le problème lié à l'Action Wrapper a été de trouver les bons paramètres à lui associer. En effet, nous avons remarqué qu'en augmentant la taille des actions prédites, les résultats se dégradaient, probablement à cause d'une surcharge d'informations ou d'une complexité accrue dans les prédictions. Ainsi, après plusieurs essais, nous avons déterminé que 2 à 3 actions constituaient le paramètre optimal pour l'Action Wrapper, offrant un bon compromis entre performance et stabilité de l'algorithme.

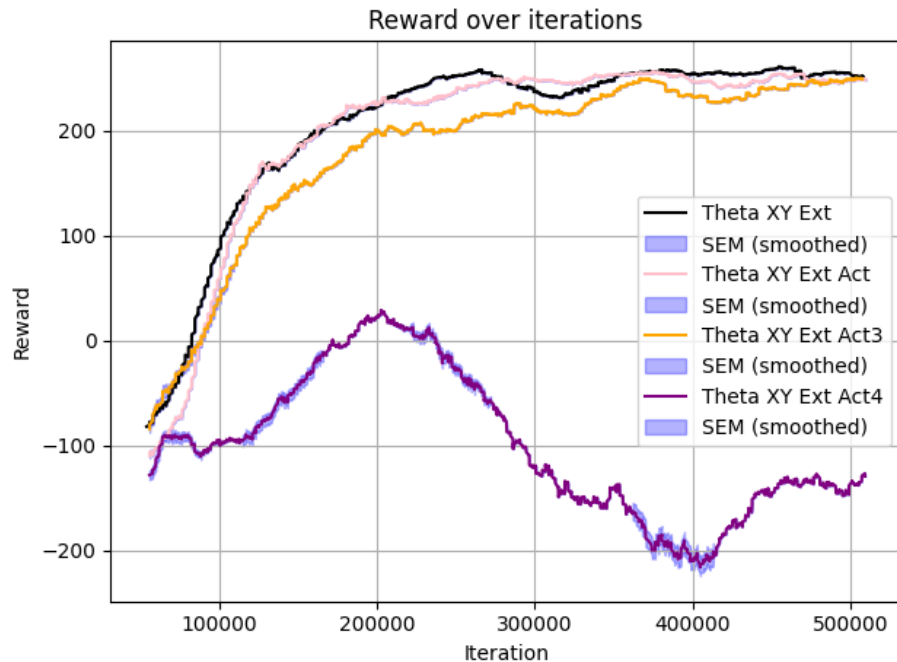


Figure 3: Valeurs moyenne des rewards selon l'action wrapper

Comme le montre la figure ci-dessus, la courbe rose, représentant l'action wrapper prédisant deux actions, affiche les meilleures performances. L'approche avec trois actions montre des résultats similaires à ceux sans action wrapper, tandis que l'approche avec quatre actions génère des performances nettement inférieures.

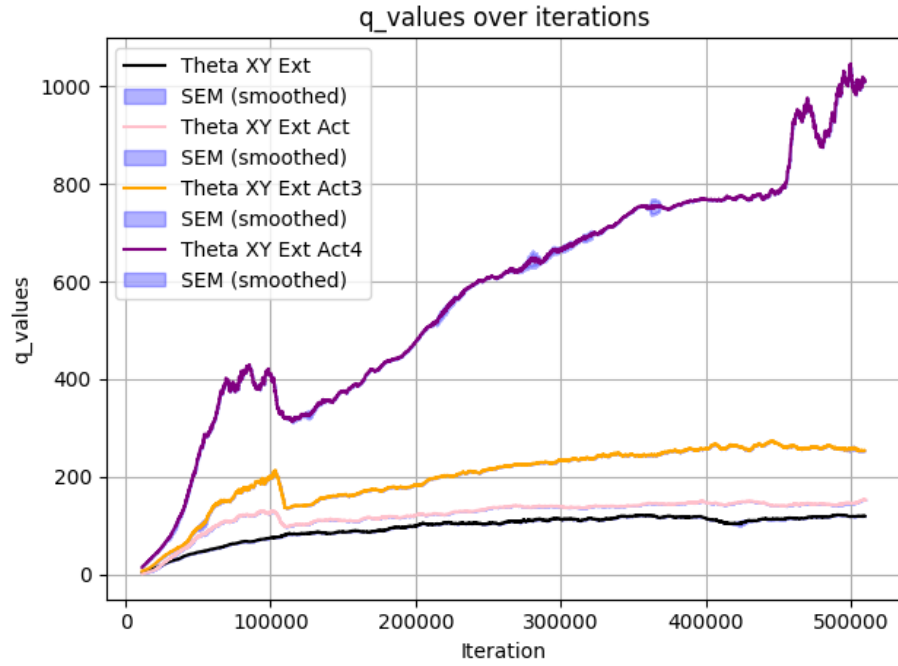


Figure 4: Valeurs moyenne des qvalues selon l'action wrapper

En approfondissant l'analyse, on observe que les valeurs Q associées à l'action wrapper prédisant quatre actions connaissent une forte explosion et augmentent de manière significative. Cette dynamique semble corrélée avec l'inefficacité observée de cette approche.

3.1 Sans mémoire

Ce cas est sûrement le moins intéressant à analyser pour autant nous avons d'assez bons résultats dessus.

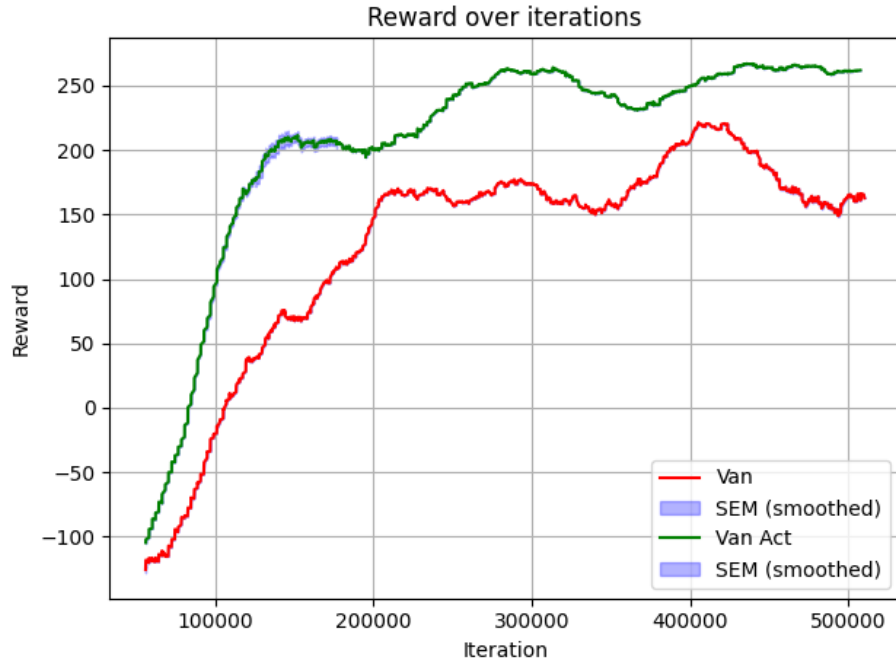


Figure 5: Valeurs moyenne des rewards (van vs van act)

Comme nous pouvons le voir, l'Action Wrapper augmente les performances de manière assez significative de l'ordre de 50 points de récompenses. Cette amélioration est clairement visible et montre l'impact positif de l'Action Wrapper sur les résultats de l'algorithme.

3.2 Avec mémoire des positions (x,y) ou sur théta

Les cas suivants montrent bien plus significativement l'impact de l'action wrapper :

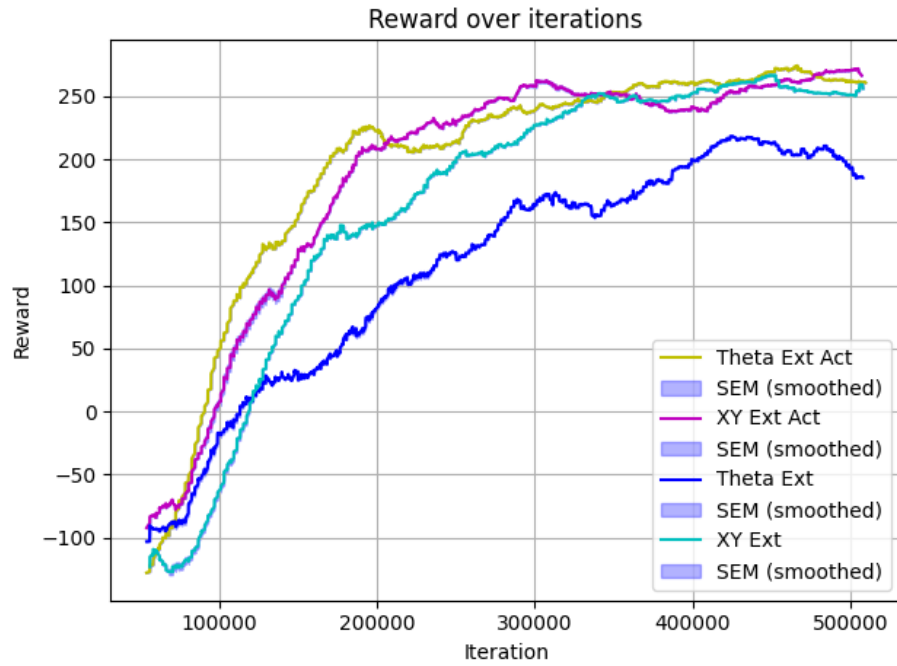


Figure 6: Valeurs moyenne des rewards (theta or xy vs act wrapper)

Ici, les courbes jaune et rose illustrent l'impact de l'ajout de l'Action Wrapper. On observe que leur évolution suit une tendance distincte par rapport aux autres courbes, ce qui met en évidence l'effet de cette modification sur les performances de l'agent. Cependant, il est à noter que la courbe cyan demeure relativement proche de son homologue avec l'Action Wrapper.

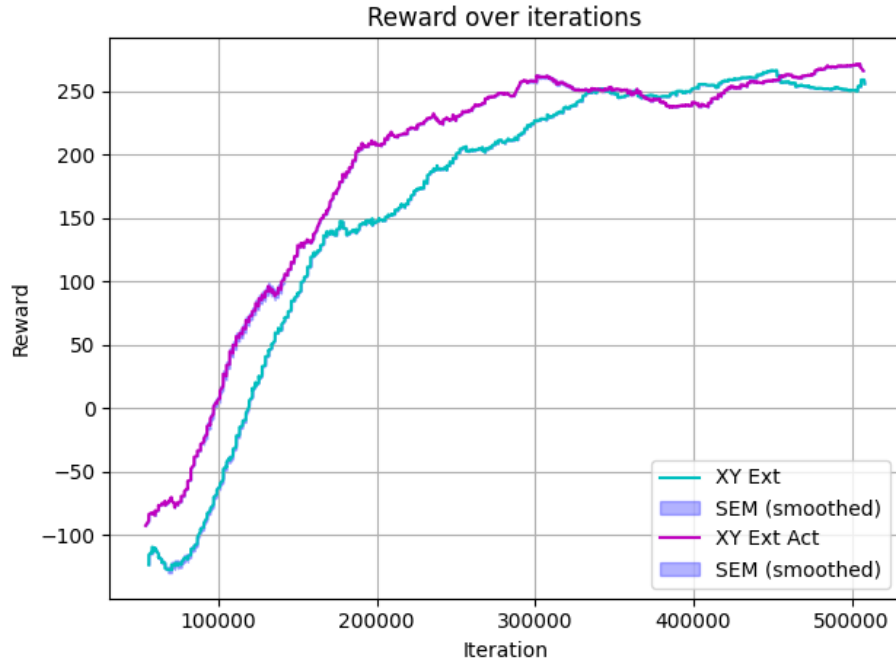


Figure 7: Valeurs moyenne des rewards (xy vs act wrapper)

Les courbes avec et sans Action Wrapper étant équivalentes, on peut en conclure que l'Action Wrapper n'a pas d'impact significatif sur les performances de l'agent dans cet environnement. Cela suggère que l'agent ne bénéficie pas de cette modification, ou que celle-ci n'est pas suffisamment optimisée pour améliorer l'apprentissage. En revanche, l'ajout de la mémoire (x,y) semble jouer un rôle crucial, comme en témoigne la différence marquée avec les autres courbes. Cela indique que la gestion de l'information spatiale par l'agent améliore nettement ses performances, ce qui pourrait être un axe d'amélioration important pour des tâches similaires.

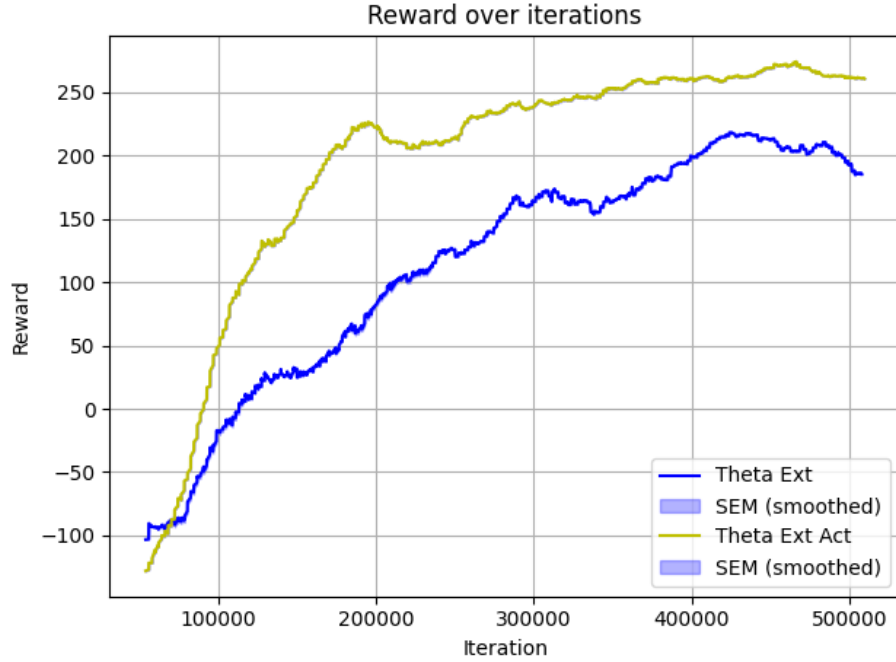


Figure 8: Valeurs moyenne des rewards (theta vs act wrapper)

Par contre, ici, dès le début, l’Action Wrapper offre de meilleures performances. On observe une progression plus rapide par rapport aux autres scénarios, ce qui suggère que son impact positif est également lié au type de mémoire apportée.

3.3 Avec mémoire des positions (x,y) et sur théta

Pour finir, nous allons analyser l’impact de l’Action Wrapper sur l’environnement partiellement observé, défini avec l’intégration de toutes les mémoires. Cette configuration permet d’évaluer comment l’Action Wrapper interagit avec une meilleure rétention d’information et dans quelle mesure il influence les performances de l’agent dans un contexte où l’observation est limitée. Nous pourrions penser qu’il entraînera de meilleurs résultats, car l’ajout de mémoire combiné à l’Action Wrapper devrait théoriquement permettre une prise de décision plus efficace. Toutefois, il reste à vérifier si cette amélioration se traduit concrètement dans les performances observées.

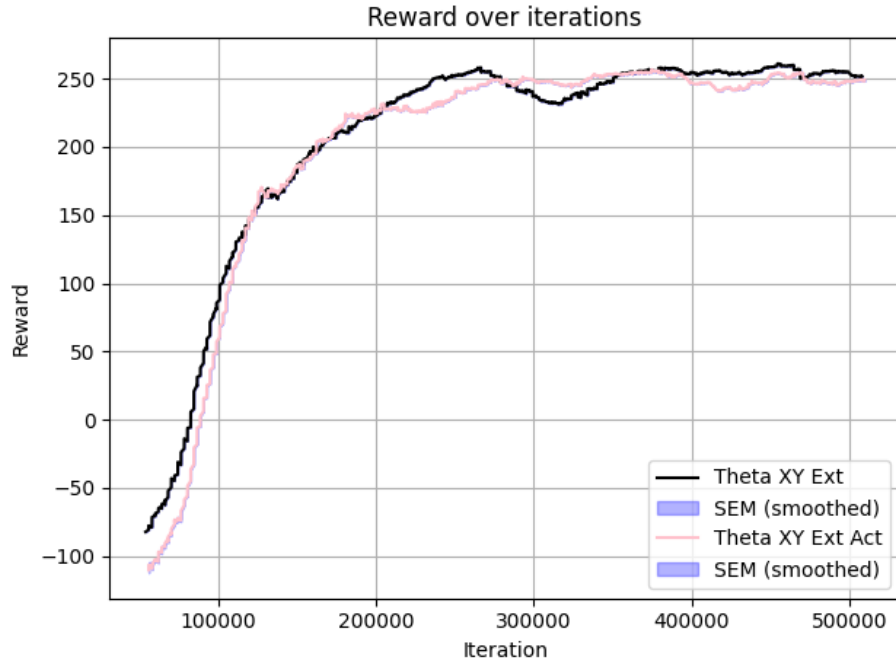


Figure 9: Valeurs moyenne des rewards (theta et xy vs act wrapper)

Le graphique montre que l'ajout de l'Action Wrapper n'apporte pas une amélioration significative dans un premier temps, les deux cas étant équivalents au début. Toutefois, vers la fin, l'Action Wrapper permet à l'agent de se démarquer légèrement, suggérant qu'il pourrait avoir un effet bénéfique à long terme, bien que cet impact ne soit pas immédiatement perceptible.

3.4 Sans wrapper

Dans ce cas-ci, nous allons étudier l'effet d'un action wrapper sur un environnement totalement observé.

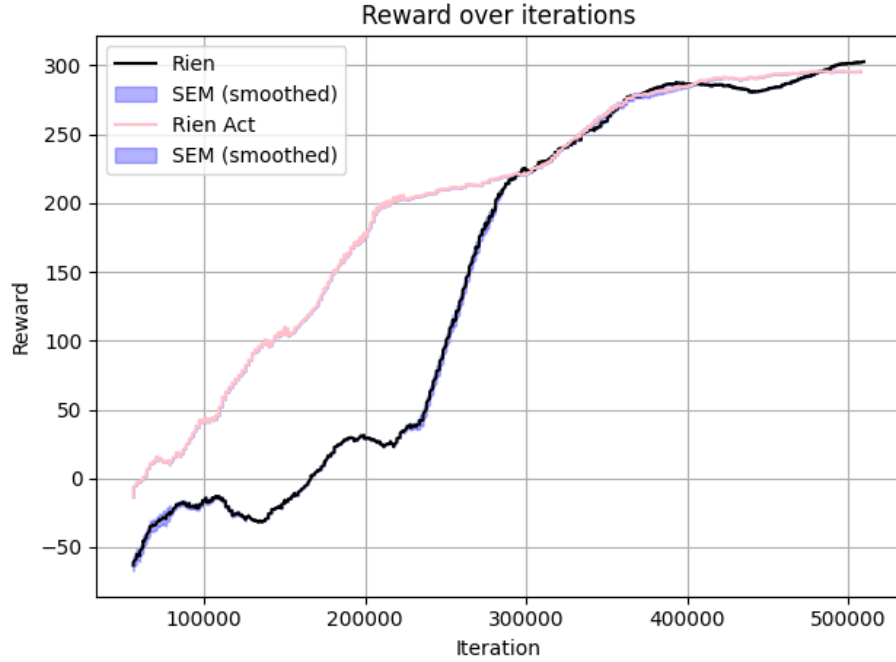


Figure 10: Valeurs moyenne des rewards (sans wrappers et avec un action wrapper)

Ces résultats mettent en évidence l'intérêt d'intégrer un action wrapper, même dans un environnement où l'état est entièrement observable. En effet, malgré l'absence de partialité dans l'observation, l'utilisation de cet outil permet à l'agent d'améliorer significativement sa vitesse de convergence. Ainsi, l'action wrapper ne se limite pas à des contextes partiellement observables, mais constitue un levier d'optimisation plus généralisable dans l'apprentissage par renforcement.

3.5 Conclusion

L'Action Wrapper peut être utile dans des environnements où l'agent dispose de très peu d'informations sur son état ou son environnement. En revanche, dans des contextes où l'agent possède déjà une grande quantité d'informations, son utilisation peut entraîner une instabilité dans l'apprentissage. De plus, chaque action prédite ajoute un coût computationnel exponentiel, rendant les calculs plus lourds. Il est donc essentiel d'effectuer un travail d'optimisation afin de trouver un équilibre entre la quantité d'informations fournies à l'agent et l'utilisation de l'Action Wrapper pour maximiser ses performances.